

MICROSERVICES COURSE

Session 4

Resilience Patterns

Circuit Breaker · Retry · Exponential Backoff · Graceful Degradation



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

Monday

3:00 PM – 5:30 PM

Online Session

2.5 Hours

Phase 1

Session 4 of 29

Phase 1 — Foundation & Core Patterns



You are here — Session 4 of 8 in Phase 1. The Gateway is secured. Today we protect the services BEHIND it.

Black Friday — One Slow Service, Total Outage

- 1 50,000 users placing orders simultaneously
- 2 Payment Service takes 10 seconds instead of 200ms
- 3 Order Service has 200 threads — all 200 now waiting
- 4 New requests arrive — no threads available
- 5 API Gateway gets no response — returns 503 to ALL users
- 6 Users retry — making it worse. Platform is down.

Root cause: ONE slow service. Not even down — just slow. This is Cascading Failure.

Partial Failure vs Cascading Failure

Partial Failure

A service is available but degraded — slow, errors on some calls

Cascading Failure

One failing service causes dependent services to fail in a chain

Fail Fast

Detect failure immediately and return error — do not wait for timeout

Fail Safe

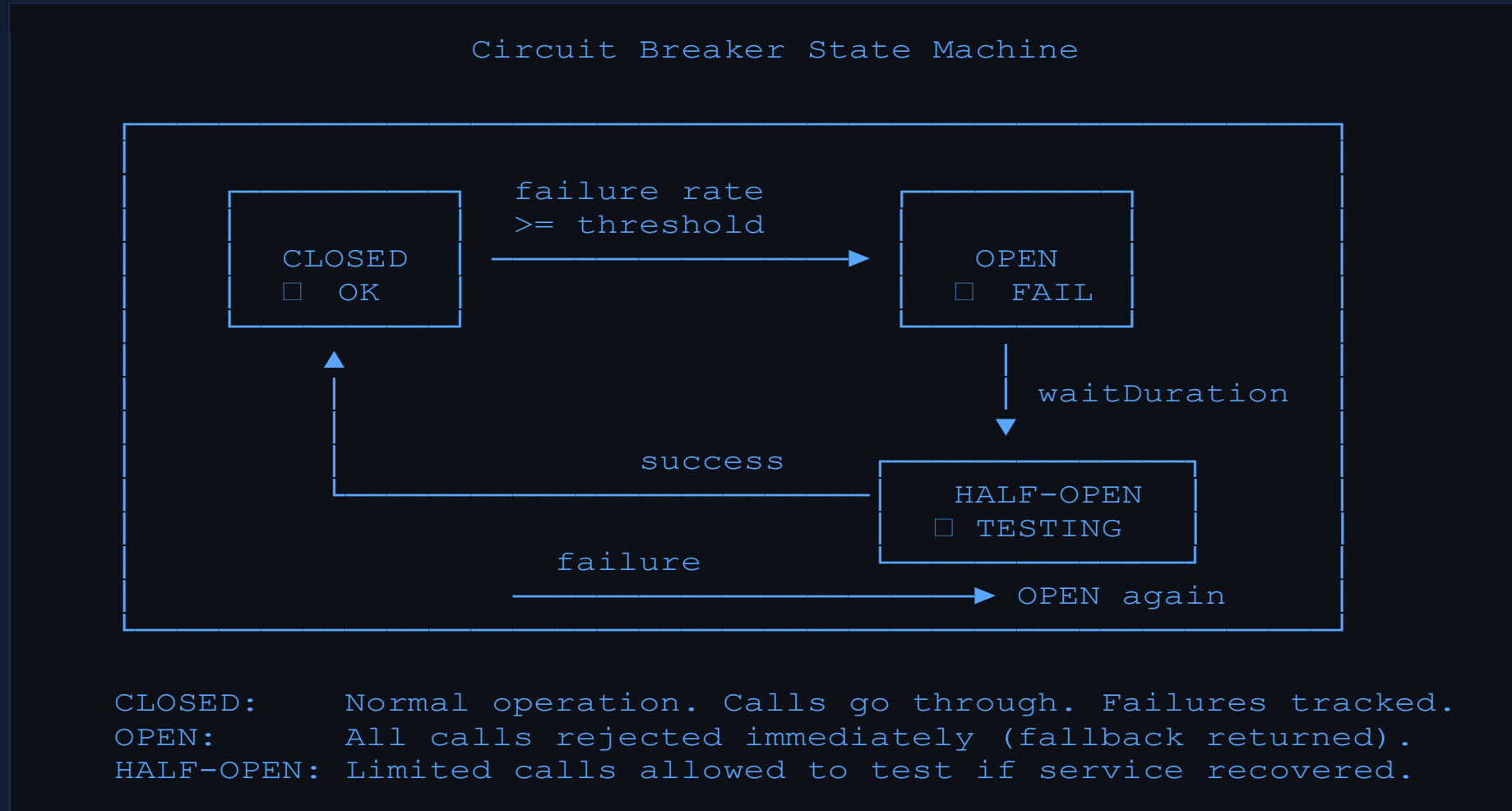
Return a degraded/cached response instead of an error

Our approach: BOTH — fail fast from Circuit Breaker + fail safe via fallback response

Resilience4j Patterns — Where Today Fits

Pattern	Problem Solved	Session
Circuit Breaker	Stops calls to a failing service — fail fast	Session 4 ← Today
Retry	Retries transient failures with backoff	Session 4 ← Today
Fallback	Returns degraded response when failure occurs	Session 4 ← Today
Bulkhead	Limits concurrent calls — isolates blast radius	Session 5
Rate Limiter	Limits calls per time period (outgoing)	Session 5
TimeLimiter	Fails fast if call exceeds timeout duration	Session 5

Circuit Breaker — The State Machine



Circuit Breaker — The State Machine



Without HALF-OPEN, the Circuit Breaker would stay OPEN forever — the service recovers but traffic never resumes.

State Transitions — Explained

CLOSED → OPEN

failureRateThreshold (e.g. 50%) exceeded in slidingWindow (e.g. last 10 calls). All calls immediately rejected.

OPEN → HALF-OPEN

After waitDurationInOpenState (e.g. 5s) — CB auto-transitions. permittedNumberOfCallsInHalfOpenState (e.g. 3) calls allowed through.

HALF-OPEN → CLOSED

All permitted calls succeed — service has recovered. Normal operation resumes, failure counters reset.

HALF-OPEN → OPEN

Any call fails in HALF-OPEN state. Back to OPEN — wait again before retrying.

Circuit Breaker — Key Configuration Properties

Property	Example	Meaning
<code>sliding-window-size</code>	10	Track the last N calls
<code>failure-rate-threshold</code>	50	Open if 50%+ of those calls fail
<code>wait-duration-in-open-state</code>	5s	Wait before trying HALF-OPEN
<code>permitted-calls-in-half-open</code>	3	Probe calls allowed in HALF-OPEN
<code>register-health-indicator</code>	true	Expose state via <code>/actuator/health</code>

Dependencies

pom.xml — Order Service

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
<!-- AOP is REQUIRED — Resilience4j annotations use Spring AOP -->
```

Circuit Breaker Configuration

application.yml – Order Service

```
resilience4j:
  circuitbreaker:
    instances:
      paymentService:
        sliding-window-type: COUNT_BASED
        sliding-window-size: 10          # track last 10 calls
        failure-rate-threshold: 50       # open if 50%+ fail
        wait-duration-in-open-state: 5s  # wait 5s before HALF-OPEN
        permitted-number-of-calls-in-half-open-state: 3
        automatic-transition-from-open-to-half-open-enabled: true
        register-health-indicator: true  # expose via Actuator

management:
  health:
    circuitbreakers:
      enabled: true
  endpoints:
    web:
      exposure:
        include: health,circuitbreakers
```

Payment Service stub with random failures

PaymentService.java — Order Service — simple stub for testing resilience

```
@Service
public class PaymentService {

    private final Random random = new Random();
    private final RestTemplate restTemplate;

    // Simulates 50% failure rate for demo purposes
    public PaymentResponse processPayment(PaymentRequest request) {
        if (random.nextInt(10) < 5) { // 50% chance
            throw new RuntimeException("Payment Service unavailable");
        }
        return new PaymentResponse("APPROVED", request.getAmount());
    }
}
```

@CircuitBreaker with Fallback

OrderService.java — Order Service

```
@Service
public class OrderService {
    @Autowired private PaymentService paymentService;
    @CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
    public OrderResponse createOrder(OrderRequest request) {
        PaymentResponse payment = paymentService.processPayment(
            new PaymentRequest(request.getAmount()));
        return new OrderResponse("CONFIRMED", payment.getTransactionId());
    }
    // Fallback: MUST match original params + Throwable as last param
    public OrderResponse paymentFallback(OrderRequest request, Throwable ex) {
        log.warn("Payment failed, returning PENDING. Reason: {}", ex.getMessage());
        return new OrderResponse("PENDING", "Will retry payment");
    }
}
```

 **Fallback signature must match the original EXACTLY + one extra Throwable parameter. If it doesn't match, Spring silently ignores it.**

Testing the Circuit Breaker — Watch It Open

Send 10+ POST requests to `/api/orders`

→ After ~5 failures: "Circuit Breaker OPEN" in logs, fallback for ALL subsequent calls

`GET /actuator/circuitbreakers`

After 5+ failures:

`state: OPEN`

Wait 5 seconds, send 1 request:

`state: HALF_OPEN`

Send 3 successful requests:

`state: CLOSED`

When Retry Helps vs Hurts

Retry HELPS

Transient network glitch (momentary timeout)

Temporary DB connection pool exhaustion

Service restarting — 2-3 seconds unavailable

DNS resolution delay

Retry HURTS

Service is overloaded — retry adds more load

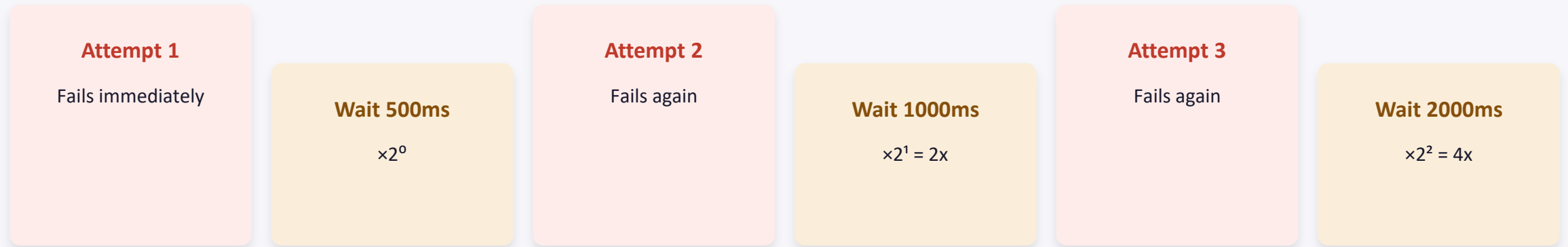
Logic error — retrying the same wrong input

Thundering herd — all retries hit at same time

Payment already processed — duplicate transaction!

 **Never retry a payment call without idempotency checks. If Payment Service processed the payment but timed out before responding, retry = double charge.**

Exponential Backoff — 500ms → 1s → 2s



```
exponential-backoff-multiplier: 2
```

```
500ms × 20 = 500ms,    500ms × 21 = 1000ms,    500ms × 22 = 2000ms
```


Configuring Retry with Backoff

`application.yml` – Order Service (add to existing config)

```
resilience4j:
  retry:
    instances:
      paymentService:
        max-attempts: 3           # try 3 times total
        wait-duration: 500ms     # wait 500ms between attempts
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2 # 500ms → 1s → 2s
        retry-exceptions:
          - java.lang.RuntimeException # only retry on these
        ignore-exceptions:
          - java.lang.IllegalArgumentException # never retry on these
```

Execution Order — CircuitBreaker Wraps Retry

OrderService.java — Order Service

```
@CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
@Retry(name = "paymentService") // Retry wraps INSIDE CircuitBreaker
public OrderResponse createOrder(OrderRequest request) { ... }

// Request flow:
// CircuitBreaker check → if OPEN: fallback immediately
//   if CLOSED: Retry wrapper
//     → attempt 1 → fail → wait 500ms
//     → attempt 2 → fail → wait 1000ms
//     → attempt 3 → fail
//   → CircuitBreaker records 3 failures → may open
//   → fallback called
```

❓ 3 retry attempts = 3 failures recorded by the CB. Design your thresholds accordingly.

Observing Retry Attempts in Logs

RetryLogger.java – Order Service

```
@Component
public class RetryLogger {
    @Autowired private RetryRegistry retryRegistry;

    @PostConstruct
    public void attachRetryListeners() {
        retryRegistry.retry("paymentService").getEventPublisher()
            .onRetry(e -> log.warn("[RETRY] Attempt #{} – {}",
                e.getNumberOfRetryAttempts(), e.getLastThrowable().getMessage()))
            .onSuccess(e -> log.info("[RETRY] Succeeded after {} attempt(s)",
                e.getNumberOfRetryAttempts()));
    }
}
```

🔗 Watch logs: *[RETRY] Attempt #1... #2... #3... then fallback. Note the 500ms/1000ms timing gaps.*

Lab 3A — Circuit Breaker & Retry on Payment Service

1 Create Payment Service with Controlled Failures

New service, port 8083, configurable failure rate

2 Apply `@CircuitBreaker` on Order Service

Fallback returns PENDING order response

3 Add Retry with Exponential Backoff

3 attempts, 500ms → 1000ms wait

4 Write Unit Tests

Minimum 3 tests covering CB + retry behaviour

Lab 3A Acceptance Criteria

- ✓ POST /api/orders → "CONFIRMED" when payment succeeds
- ✓ POST /api/orders → "PENDING" when payment fails consistently
- ✓ After 5+ failures: /actuator/circuitbreakers shows state: OPEN
- ✓ After 5s wait: state transitions to HALF_OPEN automatically
- ✓ Retry attempts visible in logs: "[RETRY] Attempt #1", "#2", "#3"
- ✓ Exponential timing visible: 500ms → 1000ms between attempts
- ✓ All 3 unit tests pass: mvn test
- ✓ Commit: session-04: add-circuit-breaker-and-retry-on-order-payment

What We Added to the Platform Today



Capability Added

Fault Tolerance & Service Resilience

Order Service fails fast, retries transient errors, and returns graceful PENDING — users are never left hanging.



Services Added / Updated

NEW: Payment Service (8083) — configurable failure rate

UPDATED: Order Service — @CircuitBreaker + @Retry

→ paymentFallback() returning PENDING

- ❓ Session 1-3: Config + Eureka + Product + Gateway (secured)
- ❓ Session 4: Payment + Order Service with Circuit Breaker + Retry

Config



Eureka



Product



Gateway



Sec.

Payment



Order



Inventory



Notif.





Daily Quiz

8 Questions · 10 Minutes

Topics: Circuit Breaker States · Fallback Methods · Retry vs Idempotency · Exponential Backoff

Checkpoint Commit & Homework

Checkpoint — Definition of Done

- ☐ CB opens after threshold — Actuator shows OPEN
- ☐ Fallback returns PENDING order
- ☐ Retry attempts in logs (#1, #2, #3)
- ☐ Exponential backoff timing correct
- ☐ Pushed: session-04: ...

Pre-Session 5 Reading (15 min)

Resilience4j — Bulkhead & TimeLimiter
resilience4j.readme.io

Knowledge Check Questions for Session 5:

- What is the difference between Bulkhead and Circuit Breaker?
- Why does TimeLimiter require CompletableFuture?
- What is thread pool exhaustion?

Common Issues & Solutions



@CircuitBreaker annotation has no effect

Missing spring-boot-starter-aop — AOP is required for Resilience4j annotations to work



Fallback method never called

Signature mismatch — must match original params + Throwable as last param, same class



CB never opens (always CLOSED)

slidingWindowSize too large for test requests — try size:5, threshold:50 for faster demo



Retry not visible in logs

Add RetryLogger @Component — verify resilience4j.retry instance name matches @Retry name



404 on /actuator/circuitbreakers

Add management.endpoints.web.exposure.include: health,circuitbreakers



Session 4 Complete

Order Service now survives Payment Service failures gracefully

Next: Session 5 — Resilience Advanced: Bulkhead & TimeLimiter · Wednesday, 3:00–5:30 PM, Online

[microservices-pro-course](#) · [microservices-pro-platform](#) · Dr. Sayed Baladoh